

Connection to GCP:

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to cs411-429620.
albertyan53@cloudshell:~ (cs411-429620)$ gcloud sql connect crimestats-uiuc-cs411 --user=root --quiet\
> bertyan53@cloudshell:~ (cs411-429620)$ gcloud sql connect crimestats-uiuc-cs411 --user=root --quiet1:~

Allowlisting your IP for incoming connection for 5 minutes...done.
Connecting to database with SQL user [root].Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 22485
Server version: 8.0.31-google (Google)

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> 
```

Creation of Tables DDL Commands:

```
CREATE TABLE Universities(
UniversityId INT,
UniversityName VARCHAR(255),
PRIMARY KEY(UniversityId)
);
```

```
CREATE TABLE City(
cityName VARCHAR(255),
stateName VARCHAR(255),
population INT,
medianAge REAL,
povertyRate REAL,
medianIncome REAL,
medianProperty REAL,
PRIMARY KEY(cityName,stateName)
);
```

```
CREATE TABLE Crime(  
    crimeId INT,  
    universityID INT,  
    crimeType VARCHAR(255),  
    crimeAddress VARCHAR(255),  
    occurredDate VARCHAR(255),  
    occurredTime VARCHAR(255),  
    reportedDate VARCHAR(255),  
    reportedTime VARCHAR(255),  
    city VARCHAR(255),  
    state VARCHAR(255),  
    PRIMARY KEY(crimeId),  
    FOREIGN KEY(city) REFERENCES City(cityName),  
    FOREIGN KEY(universityId) REFERENCES Universities(universityId),  
    FOREIGN KEY(crimeType) REFERENCES CrimeStats(crimeType)  
);
```

```
CREATE TABLE User(  
    username VARCHAR(255),  
    password VARCHAR(255),  
    address VARCHAR(255),  
    city VARCHAR(255),  
    state VARCHAR(255),  
    PRIMARY KEY(username),  
    FOREIGN KEY(city, state) REFERENCES City(cityName,stateName)  
);
```

```
CREATE TABLE CrimeStats(  
    crimeType VARCHAR(255),  
    avgAgeVic REAL,  
    avgAgePerp REAL,  
    PRIMARY KEY(crimeType)  
);
```

Three Tables with At Least 1000 Rows:

```
mysql> SELECT Count(*) FROM Crime;
+-----+
| Count(*) |
+-----+
|      1540 |
+-----+
1 row in set (0.04 sec)
```

```
mysql> SELECT Count(*) FROM User;
+-----+
| Count(*) |
+-----+
|      1011 |
+-----+
1 row in set (0.01 sec)
```

```
mysql> SELECT Count(*) FROM City;
+-----+
| Count(*) |
+-----+
|      1101 |
+-----+
1 row in set (0.00 sec)
```

Queries:

Query 1: Find Universities and Addresses for crime instances of Assault or a Sexual Assault that were reported after 5 pm.

```
SELECT u.UniversityName AS UniversityName, reportedTime
FROM Universities u JOIN Crime c ON c.universityID = u.UniversityId
WHERE c.reportedTime > '17:00:00'
AND c.crimeType IN ('Assault', 'Sexual Assault')
AND EXISTS (
    SELECT 1
    FROM Crime c2
    WHERE c2.universityID = u.UniversityId
    AND c2.reportedTime > '17:00:00'
    AND c2.crimeType IN ('Assault', 'Sexual Assault')
);
```

```
+-----+-----+
| UniversityName | reportedTime |
+-----+-----+
| 18:53:00      | florida     |
| 17:19:00      | florida     |
| 20:48:00      | florida     |
| 21:36:00      | florida     |
| 20:30:00      | florida     |
| 19:26:00      | florida     |
| 22:55:00      | florida     |
| 18:19:00      | florida     |
| 20:59:00      | florida     |
| 20:59:00      | florida     |
| 17:34:00      | florida     |
| 17:03:00      | florida     |
| 20:42:00      | florida     |
+-----+-----+
13 rows in set, 1 warning (0.01 sec)
```

Query 2: Return the number of instances of theft, trespassing, and vandalism for each hour of the day.

```
SELECT HOUR(occurredTime) AS Hour, COUNT(c.crimeId) AS CrimeCount
FROM Crime c
WHERE c.crimeType IN ('Theft', 'Trespassing', 'Vandalism')
GROUP BY Hour
ORDER BY Hour;
```

Hour	CrimeCount
0	32
1	22
2	11
3	8
4	11
5	10
6	16
7	16
8	45
9	37
10	45
11	44
12	65
13	41
14	45
15	54
16	75
17	59
18	46
19	44
20	45
21	36
22	33
23	27

24 rows in set (0.01 sec)

Query 3: Return crime types, city names, and addresses where a crime was committed in a city that has a median age of over 26 and poverty rate is over 0.2.

```
SELECT crimeId, crimeType
FROM Crime c JOIN City on c.city = City.cityName
WHERE c.city IN (SELECT cityName FROM City WHERE medianAge>26 AND
povertyRate >0.2);
LIMIT 15;
```

crimeType	cityName	crimeAddress
Theft	Champaign	59 E Green St Champaign
Theft	Champaign	512 S Third St Champaign
Fraud	Champaign	512 S Third St Champaign
Vandalism	Champaign	512 S Third St Champaign
Battery	Champaign	59 E Green St Champaign
Battery	Champaign	501 E Green St Champaign
Theft	Champaign	212 E Green St Champaign
Vandalism	Champaign	212 E Green St Champaign
Theft	Champaign	202 E Green St Champaign
Vandalism	Champaign	202 E Green St Champaign
Theft	Champaign	59 E Green St Champaign
Sexual Assault	Champaign	400 E Green St Champaign
Theft	Champaign	102 E Green St Champaign
Theft	Champaign	1117 S Oak St Champaign
Trespassing	Champaign	1117 S Oak St Champaign

15 rows in set (0.00 sec)

Query 4: Show how many Alcohol and Drug Crimes were committed before 4 pm in each city.

```
SELECT cityName, COUNT(c.crimeId) AS CrimeCount
FROM Crime c JOIN City ON City.cityName = c.city
WHERE c.crimeType IN ('Alcohol', 'Drugs') AND occurredTime < '16:00:00'
GROUP BY cityName;
```

```
+-----+-----+
| cityName | CrimeCount |
+-----+-----+
| Champaign |          11 |
| Gainesville |         129 |
| Lincoln |           4 |
+-----+-----+
3 rows in set (0.00 sec)
```

Indexing Analysis:

In this stage, we analyze the performance of four advanced SQL queries by experimenting with different indexing designs. We use the `EXPLAIN ANALYZE` command to measure the performance of each query before and after adding indices. We explore trade-offs of adding different indices to various attributes and report on the performance gains or degradations for each indexing configuration. Our analysis includes screenshots of the `EXPLAIN ANALYZE` commands and a thorough explanation of the results.

Query 1: Find Addresses that have had an Assault or a Sexual Assault since June 5th

EXPLAIN ANALYZE

SELECT crimeAddress

FROM Crime

WHERE EXISTS (SELECT 1 FROM Crime c WHERE

c.occurredDate > '2024-06-05'

AND c.crimeType IN ('Assault', 'Sexual Assault'));

```
mysql> EXPLAIN ANALYZE
-> SELECT crimeAddress
-> FROM Crime
-> WHERE EXISTS ( SELECT 1 FROM Crime c WHERE
-> c.occurredDate > '2024-06-05'
-> AND c.crimeType IN ('Assault', 'Sexual Assault') );
+-----+
| EXPLAIN
+-----+
|
+-----+
|
+-----+
| -> Inner hash join (no condition) (cost=17.71 rows=151) (actual time=0.210..0.261 rows=151 loops=1)
-> Table scan on Crime (cost=15.86 rows=151) (actual time=0.045..0.078 rows=151 loops=1)
-> Hash
-> Limit: 1 row(s) (cost=1.86 rows=1) (actual time=0.122..0.122 rows=1 loops=1)
-> Filter: (c.occurredDate > '2024-06-05') (cost=1.86 rows=1) (actual time=0.121..0.121 rows=1 loops=1)
-> Index range scan on c using crimeType over (crimeType = 'Assault') OR (crimeType = 'Sexual Assault'), with index condition: (c.crimeType in (
'Assault','Sexual Assault')) (cost=1.86 rows=3) (actual time=0.096..0.096 rows=1 loops=1)
|
+-----+
1 row in set (0.01 sec)
```

We experimented with the following indexing designs:

1. Index on `occurredDate` in the Crime table.

- CREATE INDEX idx_crime_occurred_date ON Crime(occurredDate);

```
CREATE INDEX idx_crime_occurred_date' at line 1
mysql> CREATE INDEX idx_crime_occurred_date ON Crime(occurredDate);
Query OK, 0 rows affected (0.37 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

2. Index on `crime\_type` in the Crime table.

- CREATE INDEX idx_crime_crime_type ON Crime(crimeType);

```
mysql> CREATE INDEX idx_crime_crime_type ON Crime(crimeType);
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

3. Combined index on `occurred\_date` and `crime\_type` in the Crime table.

- CREATE INDEX idx_crime_occurred_date_crime_type ON Crime(occurredDate, crimeType);

```
mysql> CREATE INDEX idx_crime_occurred_date_crime_type ON Crime(occurredDate, crimeType);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

The EXPLAIN ANALYZE command was used to measure the performance before and after adding indices.

Query 3: Find Universities where a crime was committed in a city that has a median age of over 26 and poverty rate is less than 0.2

EXPLAIN ANALYZE

SELECT crimeId

FROM Crime c JOIN City on c.city = City.cityName

WHERE c.city IN (SELECT cityName FROM City WHERE medianAge>26 AND povertyRate >0.2);

```
mysql> EXPLAIN ANALYZE
-> SELECT crimeId
-> FROM Crime c JOIN City on c.city = City.cityName
-> WHERE c.city IN (SELECT cityName FROM City WHERE medianAge>26 AND povertyRate >0.2);
+-----+-----+-----+-----+-----+-----+
| EXPLAIN |
+-----+-----+-----+-----+-----+-----+
|
+-----+-----+-----+-----+-----+-----+
| -> Nested loop inner join (cost=108.76 rows=17) (actual time=0.674..0.674 rows=0 loops=1)
|   -> Nested loop semijoin (cost=69.01 rows=17) (actual time=0.672..0.672 rows=0 loops=1)
|     -> Filter: (c.city is not null) (cost=15.85 rows=151) (actual time=0.068..0.124 rows=151 loops=1)
|       -> Covering index scan on c using city (cost=15.85 rows=151) (actual time=0.064..0.103 rows=151 loops=1)
|         -> Filter: ((City.medianAge > 26) and (City.povertyRate > 0.2)) (cost=0.03 rows=0.1) (actual time=0.003..0.003 rows=0 loops=151)
|           -> Index lookup on City using PRIMARY (cityName=c.city) (cost=0.03 rows=1) (actual time=0.003..0.003 rows=1 loops=151)
|         -> Covering index lookup on City using PRIMARY (cityName=c.city) (cost=0.25 rows=1) (never executed)
|
+-----+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

We experimented with the following indexing designs:

1. Index on `city\_name` in the CityDemographics table.

- CREATE INDEX idx_citydemographics_city_name ON City(cityName);

```
mysql> CREATE INDEX idx_citydemographics_city_name ON City(cityName);
Query OK, 0 rows affected (0.22 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

2. Index on `median\_age` in the CityDemographics table.

- CREATE INDEX idx_citydemographics_median_age ON City(medianAge);

```
mysql> CREATE INDEX idx_citydemographics_median_age ON City(medianAge);  
Query OK, 0 rows affected (0.11 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

3. Combined index on `city\_name` and `median\_age` in the CityDemographics table.

- CREATE INDEX idx_citydemographics_city_name_median_age ON City(cityName, medianAge);

```
mysql> CREATE INDEX idx_citydemographics_city_name_median_age ON City(cityName, medianAge);  
Query OK, 0 rows affected (0.09 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

The EXPLAIN ANALYZE command was used to measure the performance before and after adding indices.

Summary of Results

The selected indices provided significant performance improvements for the queries involving filtering and joins. The trade-offs in terms of storage and slightly slower insert/update operations were considered acceptable given the performance gains in query execution. The following indices were chosen as the final design:

1. `idx\_crime\_occurred\_date` on `Crime(occurred\_date)`
2. `idx\_crime\_crime\_type` on `Crime(crime\_type)`
3. `idx\_crime\_university\_id` on `Crime(university\_id)`
4. `idx\_citydemographics\_city\_name` on `CityDemographics(city\_name)`

These indices ensure optimal performance for the given queries while balancing storage and maintenance costs.